

Tests for data-generating mechanisms

Stacy Cyrille

2026-05-27

Table of contents

1	Simulate binary variables	3
2	Simulate continuous variables	16
3	Simulate time-to-event data	28
3.1	Aalen additive model	28
3.2	Cox-Aalen model	38

```
library(tidyverse)
library(MASS)
library(knitr)
library(survival)
library(survminer)
library(timereg)
library(gtsummary)

source(here::here("R", "utils.R"))
source(here::here("R", "generate_data.R"))
source(here::here("R", "performance_eval.R"))

options(scipen = 999)

# Set seed for reproducibility
set.seed(2026)
R <- 1000
```

1 Simulate binary variables

Example 1

We generate a simple example.

- Sample size: $n = 1000$
- Number of DNA pathways: $p = 3$
- Prevalence of DNA pathways: $\pi_1 = 0.1$, $\pi_2 = 0.4$, $\pi_3 = 0.6$
- Correlation matrix:

$$\Sigma = \begin{pmatrix} 1 & 0 & 0.2 \\ 0 & 1 & 0.5 \\ 0.2 & 0.5 & 1 \end{pmatrix}$$

```
n <- 1000
prev <- c(0.1, 0.4, 0.6)

Sigma <- matrix(c(1, 0, 0.2,
                  0, 1, 0.5,
                  0.2, 0.5, 1),
                3, 3)

dna1 <- data.frame(simulate_dna_pathways(n, prev, Sigma))
head(dna1)
```

	X1	X2	X3
1	0	1	1
2	0	0	0
3	0	1	1
4	0	0	0
5	0	0	0
6	0	0	0

```
# Check prevalences
colMeans(dna1)
```

	X1	X2	X3
	0.093	0.405	0.603

```
# Check dependence (Fisher's exact tests)
fisher.test(table(dna1$X1, dna1$X2))
```

Fisher's Exact Test for Count Data

```
data: table(dna1$X1, dna1$X2)
p-value = 1
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.6397796 1.6015359
sample estimates:
odds ratio
 1.016585
```

```
fisher.test(table(dna1$X1, dna1$X3))
```

Fisher's Exact Test for Count Data

```
data: table(dna1$X1, dna1$X3)
p-value = 0.00005081
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 1.628539 4.962215
sample estimates:
odds ratio
 2.780409
```

```
fisher.test(table(dna1$X2, dna1$X3))
```

Fisher's Exact Test for Count Data

```
data: table(dna1$X2, dna1$X3)
p-value < 0.00000000000000022
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 2.674120 4.802858
sample estimates:
```

odds ratio
3.573428

Example 2

We generate only uncorrelated variables.

- Sample size:

$$n = 1000$$

- Number of DNA pathways:

$$p = 3$$

- Prevalence of DNA pathways: $\pi_1 = 0.1$, $\pi_2 = 0.4$, $\pi_3 = 0.6$
- Correlation matrix:

$$\Sigma = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

```
Sigma0 <- diag(3)
dna0 <- data.frame(simulate_dna_pathways(n, prev, Sigma0))

# Check prevalences
colMeans(dna0)
```

```
      X1      X2      X3
0.103 0.397 0.598
```

```
# Check dependence (Fisher's exact tests)
fisher.test(table(dna0$X1, dna0$X2))
```

Fisher's Exact Test for Count Data

```
data:  table(dna0$X1, dna0$X2)
p-value = 0.1375
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.8886087 2.1063913
```

```
sample estimates:
odds ratio
1.369885
```

```
fisher.test(table(dna0$X1, dna0$X3))
```

Fisher's Exact Test for Count Data

```
data: table(dna0$X1, dna0$X3)
p-value = 0.8322
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.6889528 1.6638722
sample estimates:
odds ratio
1.065647
```

```
fisher.test(table(dna0$X2, dna0$X3))
```

Fisher's Exact Test for Count Data

```
data: table(dna0$X2, dna0$X3)
p-value = 0.01222
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.5478133 0.9328525
sample estimates:
odds ratio
0.7149622
```

Example 3

We simulate correlated variables grouped into blocks.

- Sample size: $n = 1000$
- Number of DNA pathways: $p = 20$
- Prevalence of DNA pathways: $\pi \sim \mathcal{U}(0.1, 0.6)$

- Block size: $b = 5$
- Number of blocks: $K = 4$
- Within-block correlations: $\rho_{k1} = 0.6$, $\rho_{k2} = 0.5$, $\rho_{k3} = 0.4$, $\rho_{k4} = 0.1$
- Between-block correlation: $\rho_B = 0.05$

```
p <- 20
```

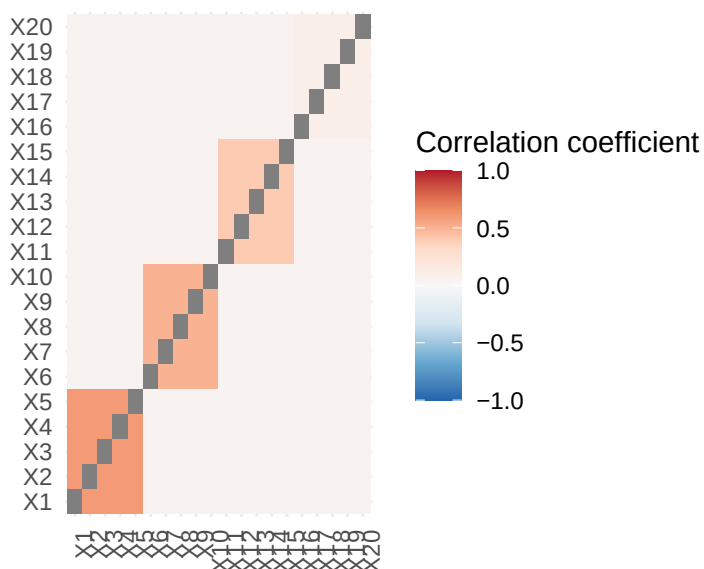
```
# Generate random prevalences
prev <- runif(20, 0.1, 0.6)
prev
```

```
[1] 0.3715163 0.3619613 0.2470627 0.4205293 0.5730484 0.3269650 0.5594099
[8] 0.5307642 0.3007579 0.4936483 0.3841484 0.5842876 0.4502982 0.3408966
[15] 0.2218836 0.2810141 0.1887874 0.2811519 0.4076672 0.5915684
```

```
# Initialize the matrix
Sigma <- generate_block_corr(p,
                             block_size = 5,
                             rho_within = c(0.6, 0.5, 0.4, 0.1),
                             rho_between = 0.05)

# Inspect the matrix
colnames(Sigma) <- paste0("X", 1:p)
rownames(Sigma) <- paste0("X", 1:p)
Sigma %>%
  as.data.frame()%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X", 1:p)),
         var2 = factor(var2, levels = paste0("X", 1:p)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1, 1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "True correlation matrix in the simulation design")
  }
```

True correlation matrix in the simulation des



```
# Simulate correlated data using this matrix
dna_block <- data.frame(simulate_dna_pathways(n, prev, Sigma))

# Check marginal prevalences
data.frame(true_prev = prev,
            est_prev = colMeans(dna_block))%>%
  rownames_to_column("variable")%>%
  kable(format = "html", caption = "True and empirical prevalences",
        col.names = c("Variable", "True prevalence", "Estimated prevalence"))
```

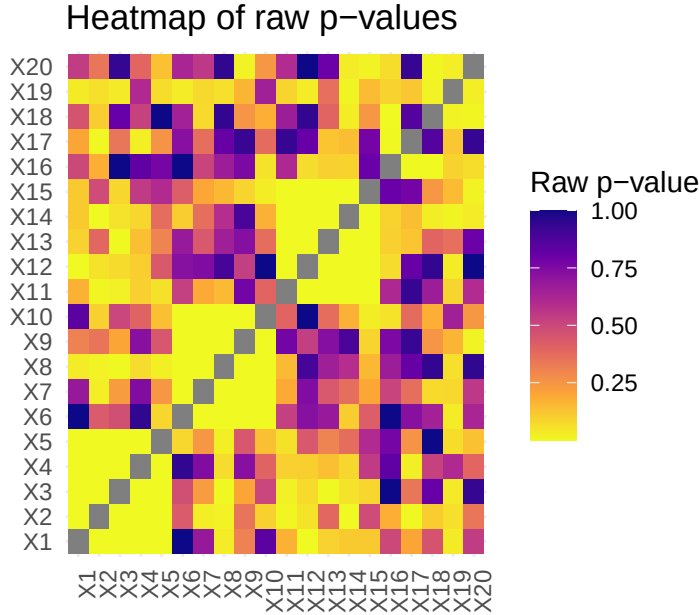
Table 1: True and empirical prevalences

Variable	True prevalence	Estimated prevalence
X1	0.3715163	0.340
X2	0.3619613	0.361
X3	0.2470627	0.243
X4	0.4205293	0.420
X5	0.5730484	0.566
X6	0.3269650	0.311
X7	0.5594099	0.579
X8	0.5307642	0.537
X9	0.3007579	0.308

Variable	True prevalence	Estimated prevalence
X10	0.4936483	0.486
X11	0.3841484	0.396
X12	0.5842876	0.572
X13	0.4502982	0.475
X14	0.3408966	0.320
X15	0.2218836	0.237
X16	0.2810141	0.268
X17	0.1887874	0.192
X18	0.2811519	0.282
X19	0.4076672	0.418
X20	0.5915684	0.598

```
# 20x20 matrix of raw p-values
pval_block <- pairwise_fisher(dna_block, names(dna_block))

# Heatmap of raw p-values
mat_long <- as.data.frame(as.table(pval_block$p.value.matrix))%>%
  mutate(Freq = ifelse(Var1==Var2, NA, Freq))
ggplot(mat_long, aes(Var1, Var2, fill = Freq)) +
  geom_tile() +
  scale_fill_viridis_c(option = "C", direction = -1) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
  labs(fill = "Raw p-value", x = "", y = "",
       title = "Heatmap of raw p-values")
```



All simulations were repeated across $R = 1000$ replications.

For each replicate, pairwise associations between DNA-based pathways were tested using Fisher's exact test, and the empirical type I error was estimated as the proportion of replications with p-values below the nominal significance level ($\alpha = 0.05$)

We simulate correlated variables grouped into blocks.

- Sample size: $n = 1000$
- Number of DNA pathways: $p = 20$
- Prevalence of DNA pathways: $\pi \sim \mathcal{U}(0.1, 0.6)$
- Block size: $b = 5$
- Number of blocks: $K = 4$
- Within-block correlations: $\rho_{k1} = 0.6$, $\rho_{k2} = 0.5$, $\rho_{k3} = 0.4$, $\rho_{k4} = 0.05$
- Between-block correlation: $\rho_B = 0.01$

```
n <- 1000
p <- 20
prev <- runif(p, 0.1, 0.6) %>%
  set_names(paste0("X", 1:p))
Sigma <- generate_block_corr(p,
```

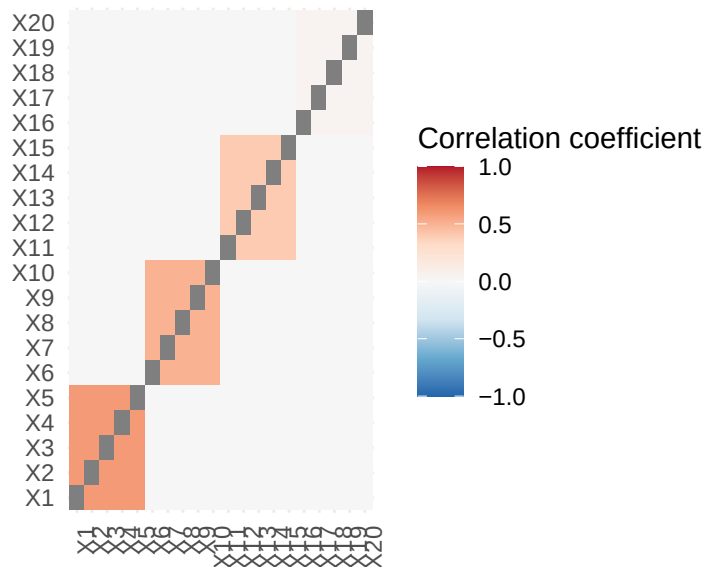
```

        block_size = 5,
        rho_within = c(0.6, 0.5, 0.4, 0.05),
        rho_between = 0.01)

# Inspect the matrix
colnames(Sigma) <- paste0("X",1:20)
rownames(Sigma) <- paste0("X",1:20)
Sigma %>%
  as.data.frame()%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "True correlation matrix in the simulation design")
  }

```

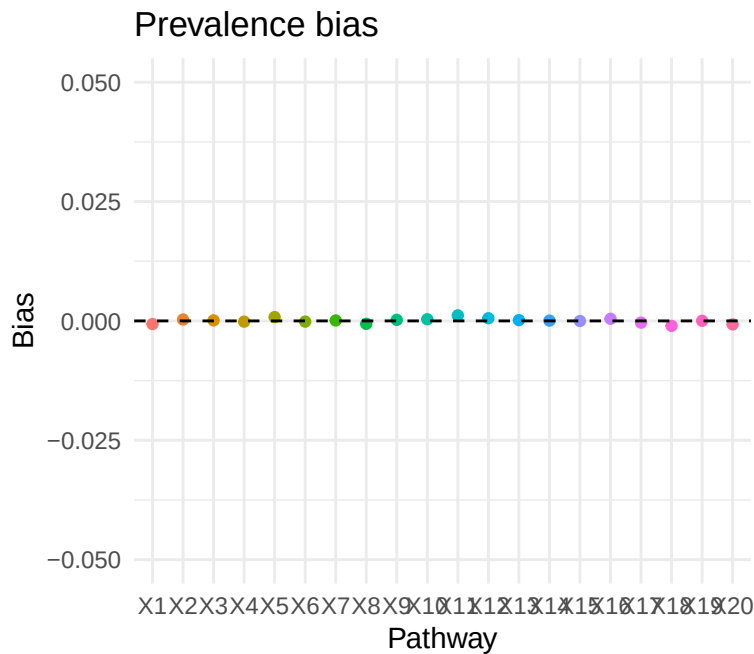
True correlation matrix in the simulation des



```
# Run simulations
dna_sims <- replicate(R,
  simulate_dna_pathways(n, prev, Sigma),
  simplify = FALSE) %>%
  imap(~ mutate(data.frame(.x), R = .y)) %>%
  bind_rows()

# Prevalence bias
dna_sims %>%
  bind_rows() %>%
  pivot_longer(-R, names_to = "pathway", values_to = "est_prev") %>%
  dplyr::full_join(rownames_to_column(data.frame(true_prev = prev),
    "pathway"), by = "pathway") %>%
  group_by(pathway) %>%
  summarize(bias = estimate_bias(true = true_prev, est = est_prev)) %>%
  mutate(pathway = factor(pathway, levels = paste0("X", 1:20))) %>%
  {
    ggplot(., aes(x = pathway, y = bias, colour = pathway)) +
      geom_point(aes(fill = "pathway")) +
      geom_hline(yintercept = 0, linetype = "dashed") +
      scale_y_continuous(limits = c(-0.05, 0.05)) +
      labs(x = "Pathway", y = "Bias", title = "Prevalence bias") +
      theme_minimal() +
  }
```

```
theme(legend.position = "none")
}
```



```
# Type I error
dna_sims_pval <- dna_sims %>%
  group_split(R) %>%
  map(~.x %>%
    dplyr::select(-R)%>%
    pairwise_fisher(vars = names()))

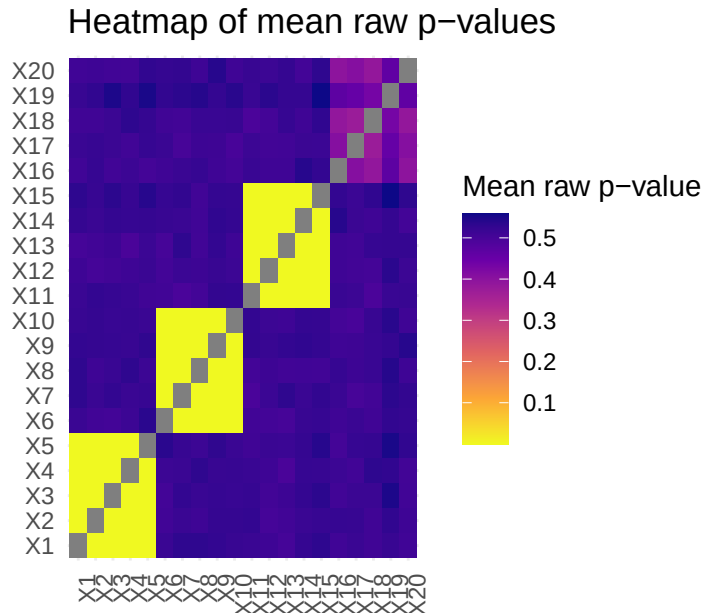
# Heatmap of mean raw p-values
dna_sims_pval %>%
  map(~as.data.frame(as.table(.x$p.value.matrix))%>%
    mutate(Freq = ifelse(Var1==Var2, NA, Freq))%>%
    reduce(bind_rows, .id = "R")%>%
    group_by(Var1, Var2)%>%
    summarise(mean_p.value = mean(Freq), .groups = "drop")%>%
    {
      ggplot(., aes(Var1, Var2, fill = mean_p.value)) +
        geom_tile() +
        scale_fill_viridis_c(option = "C", direction = -1) +
        theme_minimal() +

```

```

    theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
    labs(fill = "Mean row p-value", x = "", y = "",
         title = "Heatmap of mean row p-values")
}

```

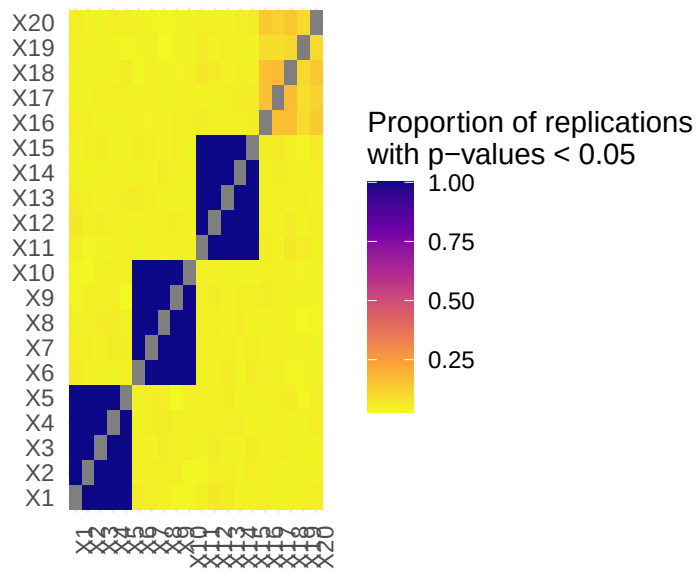


```

# Heatmap of the proportion of replications with p-values < 0.05
dna_sims_pval %>%
  map(~as.data.frame(as.table(.x$p.value.matrix))%>%
      mutate(Freq = ifelse(Var1==Var2, NA, Freq))%>%
      reduce(bind_rows, .id = "R")%>%
      group_by(Var1, Var2)%>%
      summarise(mean_typeI = mean(Freq < 0.05, na.rm = TRUE),
                .groups = "drop")%>%
  {
    ggplot(., aes(Var1, Var2, fill = mean_typeI)) +
      geom_tile() +
      scale_fill_viridis_c(option = "C", direction = -1) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Proportion of replications \nwith p-values < 0.05", x = "", y = "",
           title = "Heatmap of the proportion of replications with p-values < 0.05")
  }

```

Heatmap of the proportion of replications w



2 Simulate continuous variables

Example 1

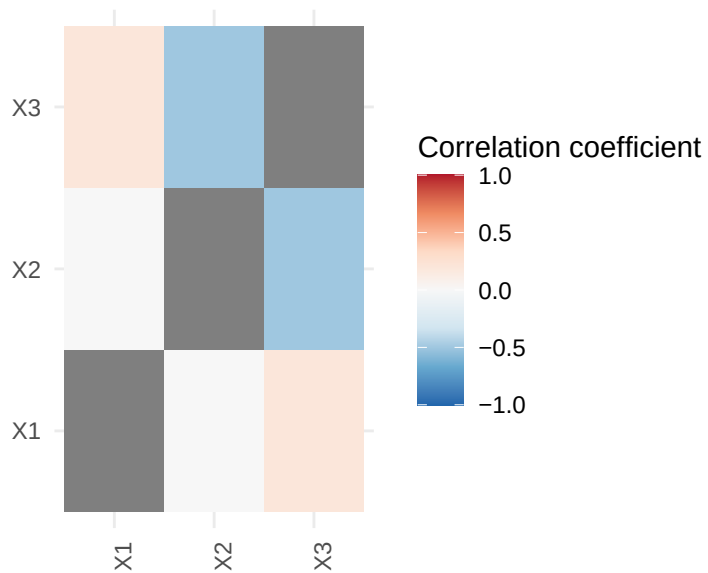
We generate a simple example.

- Sample size: $n = 1000$
- Number of RNA pathways: $p = 3$
- Correlation matrix:

$$\Sigma = \begin{pmatrix} 1 & 0 & 0.2 \\ 0 & 1 & 0.5 \\ 0.2 & 0.5 & 1 \end{pmatrix}$$

```
p <- 3
mu <- 0
Sigma <- matrix(c(1, 0, 0.2,
                  0, 1, -0.5,
                  0.2, -0.5, 1),
                3, 3)
# Inspect the matrix
colnames(Sigma) <- paste0("X",1:p)
rownames(Sigma) <- paste0("X",1:p)
Sigma %>%
  as.data.frame()%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:p)),
         var2 = factor(var2, levels = paste0("X",1:p)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "True correlation matrix in the simulation design")
  }
```

True correlation matrix in the simulation desi



```
rna1 <- simulate_rna_pathways(n, p, mu, Sigma)
```

```
# Check mean
colMeans(rna1)
```

```

              X1                      X2
0.000000000000000017721935 -0.000000000000000016462526
              X3
-0.000000000000000004572731
```

```
# Check variance
apply(rna1, 2, var)
```

```

X1 X2 X3
1  1  1
```

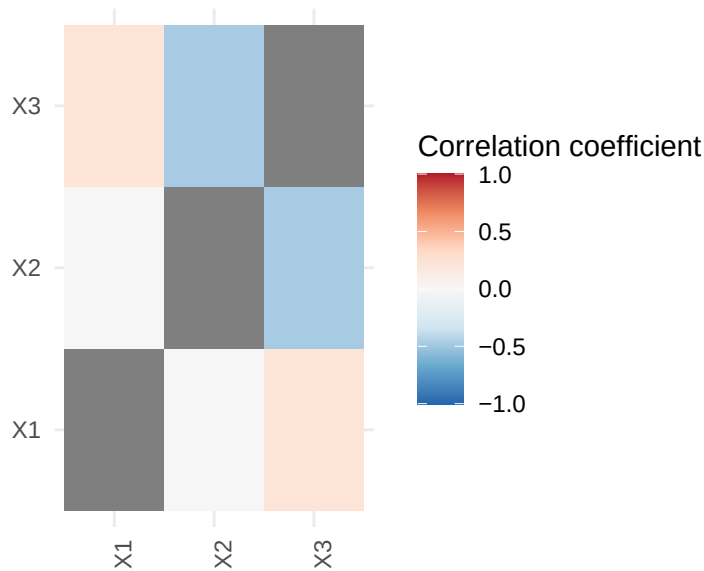
```
# Correlation matrix
as.data.frame(cor(rna1))%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
```

```

var1 = factor(var1, levels = paste0("X",1:20)),
var2 = factor(var2, levels = paste0("X",1:20)))%>%
{
  ggplot(., aes(var1, var2, fill = value)) +
    geom_tile() +
    scale_fill_distiller(palette = "RdBu", direction = -1,
                        limits = c(-1,1)) +
    theme_minimal() +
    theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
    labs(fill = "Correlation coefficient", x = "", y = "",
         title = "Correlation matrix of simulated variables")
}

```

Correlation matrix of simulated variables



Example 2

We simulate correlated variables grouped into blocks.

- Sample size: $n = 1000$
- Number of RNA pathways $p = 20$
- Block size: $b = 5$

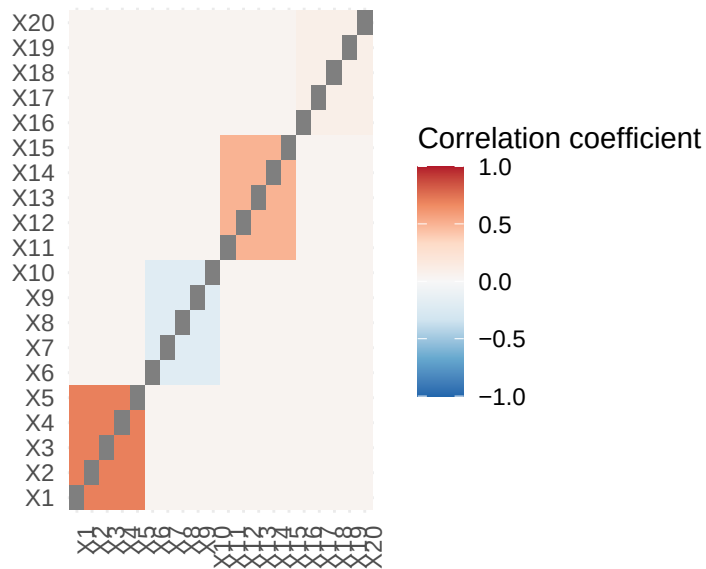
- Number of blocks: $K = 4$
- Within-block correlations: $\rho_{k1} = 0.7$, $\rho_{k2} = -0.2$, $\rho_{k3} = 0.5$, $\rho_{k4} = 0.1$
- Between-block correlation: $\rho_B = 0.05$

```
p = 20

# Initialize the matrix
Sigma <- generate_block_corr(p,
                             block_size = 5,
                             rho_within = c(0.7, -0.2, 0.5, 0.1),
                             rho_between = 0.05)

# Inspect the matrix
colnames(Sigma) <- paste0("X",1:20)
rownames(Sigma) <- paste0("X",1:20)
Sigma %>%
  as.data.frame()%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                           limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "True correlation matrix in the simulation design")
  }
```

True correlation matrix in the simulation des



```
# Simulate correlated data using this matrix
rna_block <- simulate_rna_pathways(n, p, mu, Sigma)

# Check mean
colMeans(rna_block)
```

X1	X2
0.000000000000000019128796	0.000000000000000016651611
X3	X4
0.000000000000000010266094	-0.0000000000000000033302354
X5	X6
0.000000000000000009903536	-0.0000000000000000014625454
X7	X8
-0.0000000000000000019990953	0.0000000000000000010843756
X9	X10
-0.0000000000000000023961735	0.0000000000000000004299512
X11	X12
-0.000000000000000002105087	-0.0000000000000000003363195
X13	X14
0.000000000000000001595078	-0.0000000000000000007872175
X15	X16
0.0000000000000000021785525	-0.00000000000000000014748619

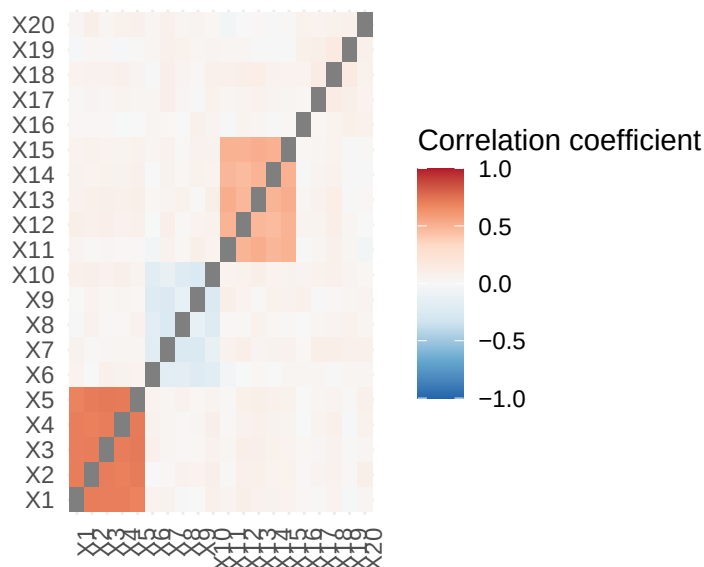
	X17	X18
	0.000000000000000003472049	-0.0000000000000000014085955
	X19	X20
	0.0000000000000000005715046	0.0000000000000000010665080

```
# Check variance
apply(rna_block, 2, var)
```

X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14	X15	X16	X17	X18	X19	X20
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

```
# Correlation matrix
as.data.frame(cor(rna_block))%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "Correlation matrix of simulated variables")
  }
```

Correlation matrix of simulated variables



All simulations were repeated across $R = 1000$ replications.

For each pathway, we computed the empirical mean, variance, and pairwise correlation coefficients of the RNA pathways across simulation replicates. Mean, variance and correlation coefficients were averaged across replicates.

- Sample size: $n = 1000$
- Number of RNA pathways $p = 20$
- Block size: $b = 5$
- Number of blocks: $K = 4$
- Within-block correlations: $\rho_{k1} = 0.7$, $\rho_{k2} = -0.2$, $\rho_{k3} = 0.5$, $\rho_{k4} = 0.3$
- Between-block correlation: $\rho_B = 0.1$

```
p <- 20
mu <- 0

# Initialize the matrix
Sigma <- generate_block_corr(p,
                             block_size = 5,
                             rho_within = c(0.7, -0.2, 0.5, 0.3),
                             rho_between = 0.1)
```

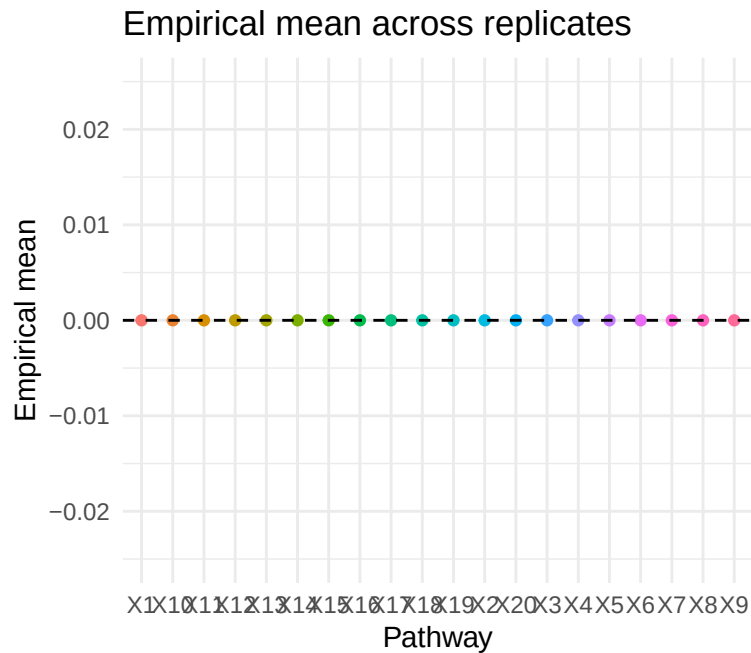
```

colnames(Sigma) <- paste0("X",1:20)
rownames(Sigma) <- paste0("X",1:20)

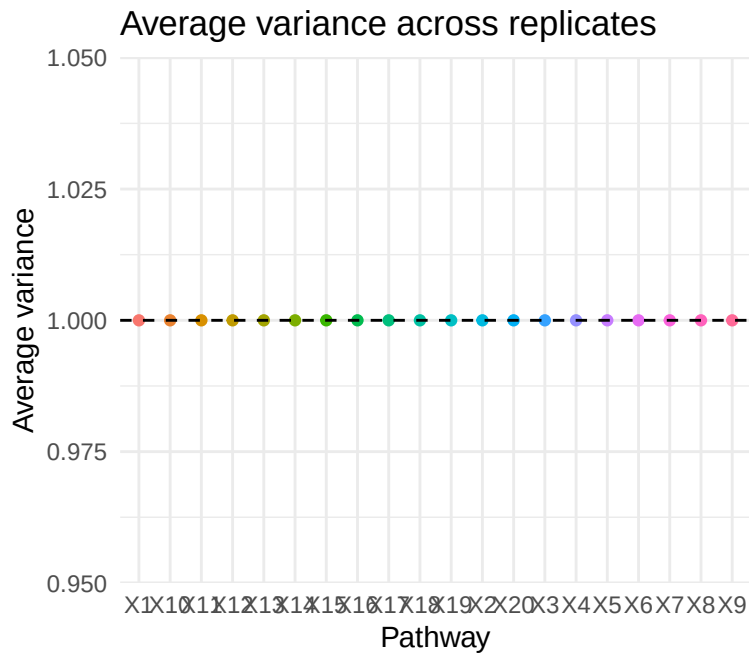
# Run simulations
rna_sims <- replicate(R,
                      simulate_rna_pathways(n, p, mu, Sigma),
                      simplify = FALSE)

# Check mean
rna_sims %>%
  map(~colMeans(.x))%>%
  reduce(bind_rows)%>%
  summarise_all(mean)%>%
  pivot_longer(everything())%>%
  {
    ggplot(., aes(x = name, y = value, colour = name))+
      geom_point()+
      geom_hline(yintercept = 0, linetype = "dashed")+
      scale_y_continuous(limits = c(-0.025, 0.025))+
      labs(x = "Pathway", y = "Empirical mean",
           title = "Empirical mean across replicates")+
      theme_minimal()+
      theme(legend.position = "none")
  }

```

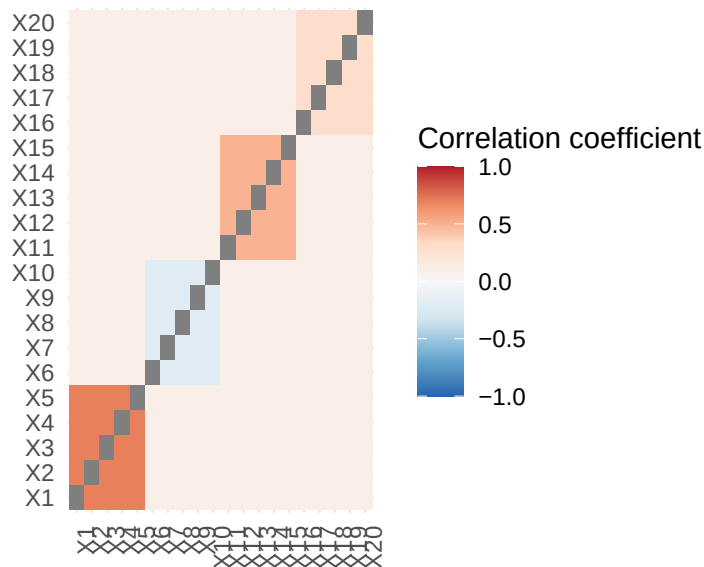


```
# Check variance
rna_sims %>%
  map(~ apply(., 2, var))%>%
  reduce(bind_rows)%>%
  summarise_all(mean)%>%
  pivot_longer(everything())%>%
  {
    ggplot(., aes(x = name, y = value, colour = name))+
      geom_point()+
      geom_hline(yintercept = 1, linetype = "dashed")+
      labs(x = "Pathway", y = "Average variance",
           title = "Average variance across replicates")+
      theme_minimal()+
      theme(legend.position = "none")
  }
```



```
# True correlation matrix
Sigma %>%
  as.data.frame()%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "True correlation matrix in the simulation design")
  }
```

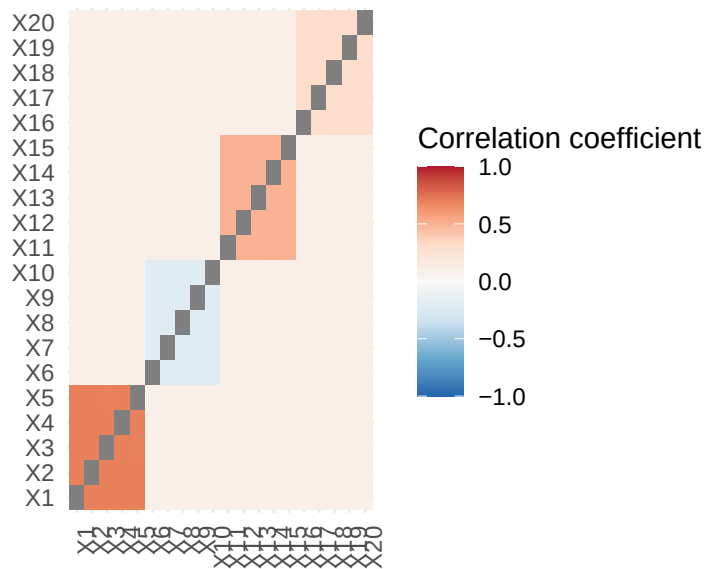
True correlation matrix in the simulation des



```
# Average correlation matrix across simulation replicates
cor_sims_rna <- rna_sims %>%
  map(cor) %>%
  reduce(`+`) / length(rna_sims)

as.data.frame(cor_sims_rna)%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "Average correlation matrix across simulation replicates")
  }
```

Average correlation matrix across simulation



3 Simulate time-to-event data

3.1 Aalen additive model

Four binary covariates were generated *independently* with prevalence of 50%, $X_j \sim \mathcal{B}(0.4)$.

Events times were simulated under the following time-constant additive hazards model:

$$\lambda(t|x) = \lambda_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4$$

with $\lambda_0 = 0.05$ and $\beta = (0.06, 0.02, 0.005, 0)$.

This means:

- Having $X_1 = 1$ increases the hazard by 0.06 compared with $X_1 = 0$
- Having $X_2 = 1$ increases the hazard by 0.02 compared with $X_2 = 0$
- Having $X_3 = 1$ increases the hazard by 0.005 compared with $X_3 = 0$
- There are no difference in hazard between $X_4 = 1$ and $X_4 = 0$

Interpretation:

- Each coefficient represents an absolute increase in instantaneous event rate
- Effects combine by addition
- A coefficient equal to zero means the variable does not contribute to the hazard

Independent censoring times were generated as $C \sim \text{Uniform}(0, c_{\max})$. The parameter $c_{\max}$ was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 5%, and the calibrated value was then fixed across all simulation replicates.

```
# Generate data
n <- 1000
p <- 4
sigma <- diag(x = 1, nrow = p, ncol = p)
# prev <- c(0.005, 0.02, 0.05, 0.20)
prev <- rep(0.4, 4)
beta <- c(0.06, 0.02, 0.005, 0)
lambda0 <- 0.05
target_cens <- 0.05
c_max <- calibrate_c_max_aalen(lambda0      = lambda0,
                               beta         = beta,
                               prev         = prev,
                               sigma        = sigma,
                               target_cens  = target_cens,
```

```

                                seed      = 2026)
c_max

```

```
[1] 273.8113
```

```

data_aalen <- simulate_aalen_data(n, prev, sigma, lambda0, beta, c_max)
head(data_aalen)

```

```

      time event X1 X2 X3 X4
1  6.199056     1  1  0  0  1
2  2.596458     1  1  0  1  1
3  7.946940     1  0  1  0  1
4  2.210118     1  1  1  1  0
5  8.175295     1  0  0  0  1
6 23.579391     1  0  0  0  0

```

```

# Check prevalences
round(colMeans(data_aalen %>% dplyr::select(starts_with("X"))), 3)

```

```

      X1      X2      X3      X4
0.404 0.383 0.424 0.399

```

```

# Check censoring rate
mean(1 - data_aalen$event)

```

```
[1] 0.046
```

Kaplan-Meier

```

km_aalen_fit <- survfit(Surv(time, event)~1, data_aalen)
km_aalen_fit

```

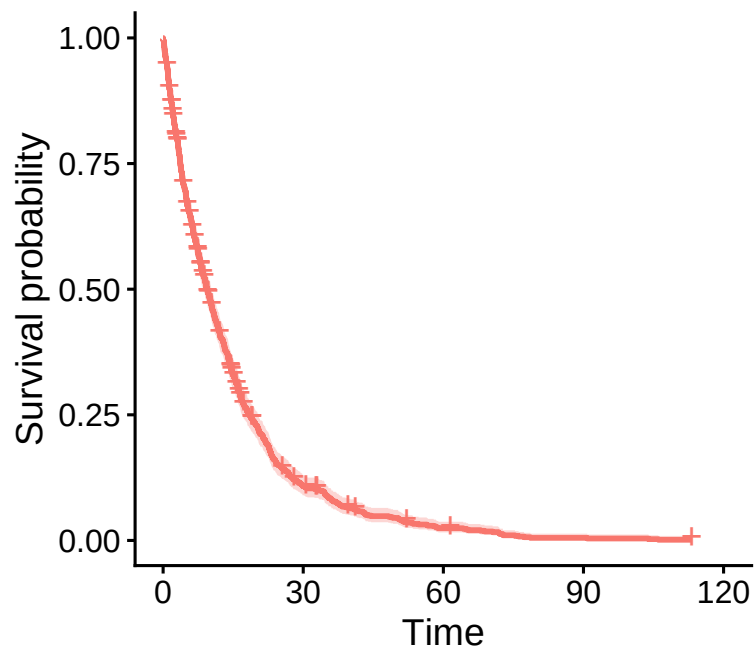
```
Call: survfit(formula = Surv(time, event) ~ 1, data = data_aalen)
```

```

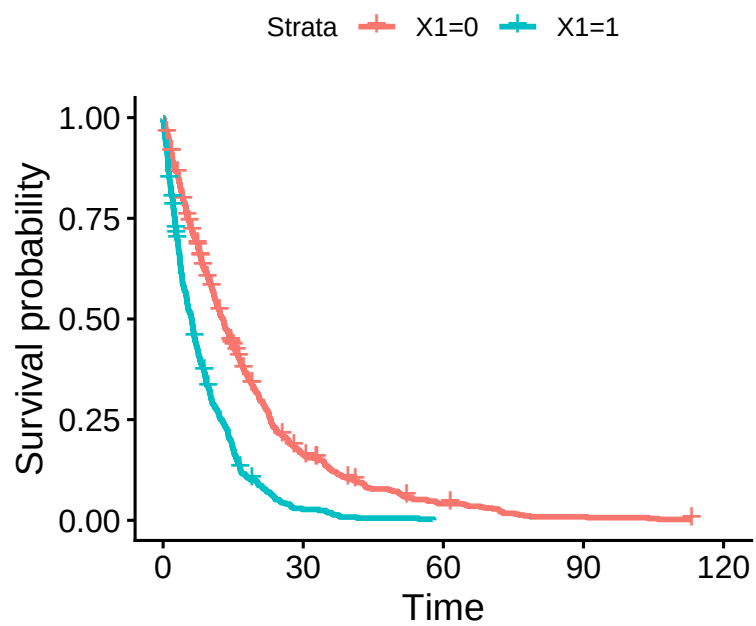
      n events median 0.95LCL 0.95UCL
[1,] 1000    954   9.49    8.49   10.4

```

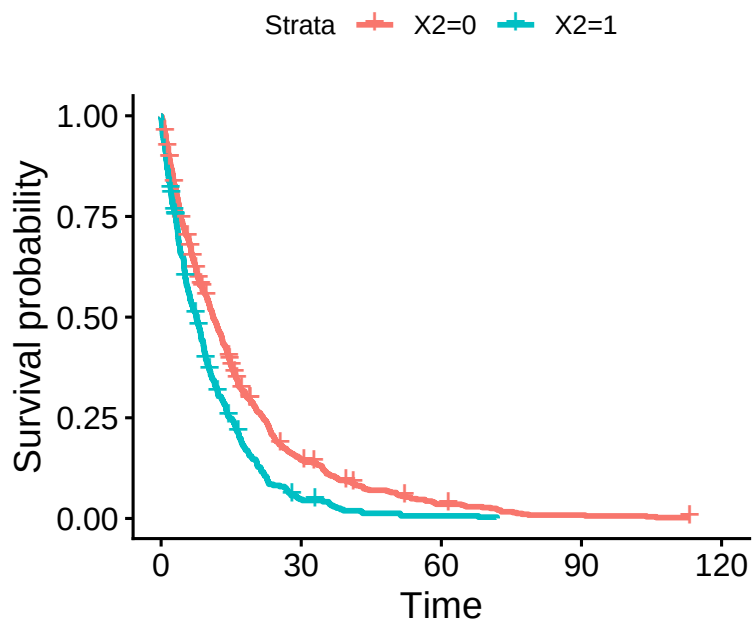
```
ggsurvplot(km_aalen_fit, data = data_aalen, legend = "none")
```



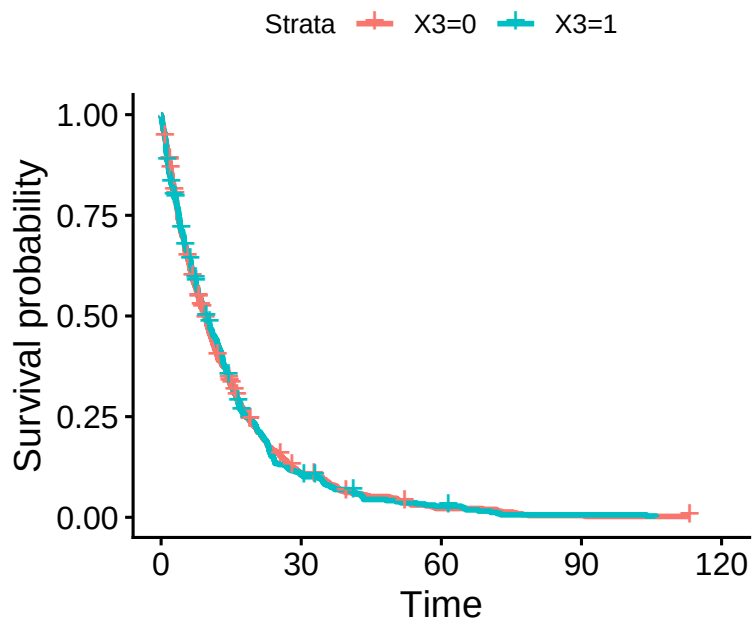
```
ggsurvplot(survfit(Surv(time, event)~X1, data_aalen))
```



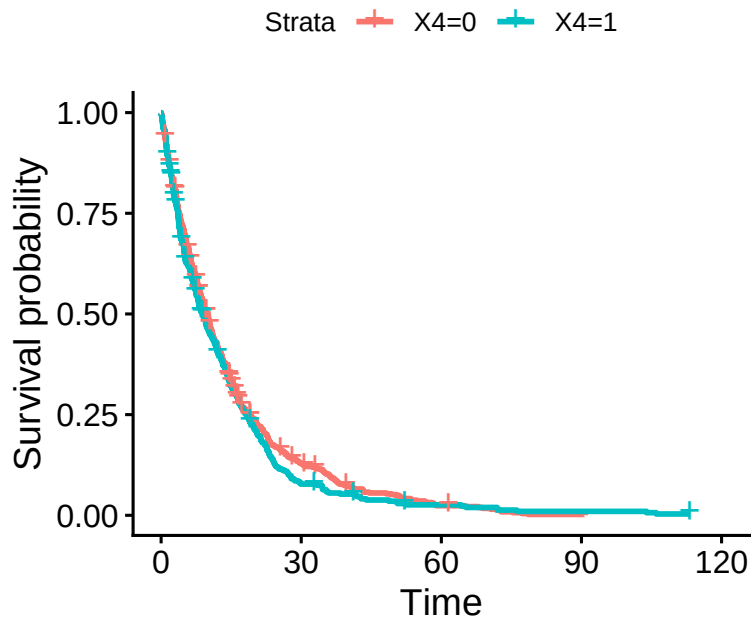
```
ggsurvplot(survfit(Surv(time, event)~X2, data_aalen))
```



```
ggsurvplot(survfit(Surv(time, event)~X3, data_aalen))
```



```
ggsurvplot(survfit(Surv(time, event)~X4, data_aalen))
```



Aalen model

```
# Fit the model
aalen_fit <- timereg::aalen(Surv(time, event)~X1+X2+X3+X4, data = data_aalen)
summary(aalen_fit)
```

Additive Aalen Model

Test for nonparametric terms

Test for non-significant effects

	Supremum-test of significance	p-value	H ₀ : B(t)=0
(Intercept)	10.20	0.000	
X1	8.04	0.000	
X2	5.09	0.000	
X3	1.50	0.940	
X4	2.41	0.326	

Test for time invariant effects

Kolmogorov-Smirnov test p-value H₀:constant effect

(Intercept)	1.200	0.000
X1	1.600	0.000
X2	1.070	0.002
X3	0.249	0.773
X4	0.490	0.081
Cramer von Mises test p-value H_0:constant effect		
(Intercept)	45.300	0.000
X1	60.700	0.000
X2	25.500	0.001
X3	0.641	0.822
X4	2.880	0.148

```
Call:
timereg::aalen(formula = Surv(time, event) ~ X1 + X2 + X3 + X4,
  data = data_aalen)
```

Test for non-significant effects

The model estimates the cumulative effects

$$B_j(t) = \int_0^t \beta_j(u) du.$$

The reported *Supremum test* evaluates $H_0 : B_j(t) = 0$, meaning the covariate has no effect on the hazard over time.

Interpretation of the results

- Intercept: $p < 0.001 \rightarrow$ Baseline hazard is clearly non zero
- X_1 : $p < 0.001 \rightarrow$ Strong evidence that X_1 affects the hazard
- X_2 : $p < 0.001 \rightarrow$ Strong evidence that X_2 affects the hazard
- X_3 : $p > 0.9 \rightarrow$ No evidence that X_3 affects the hazard
- X_4 : $p = 0.3 \rightarrow$ No evidence that X_4 affects the hazard

Test for time invariant effects

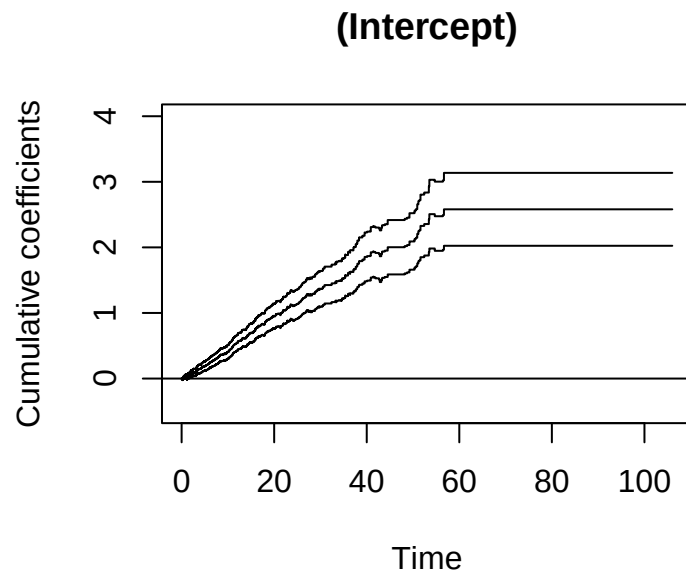
Interpretation of the results of the Kolmogorov–Smirnov test

Small p-values *rigtharrow* evidence of time-varying effects.

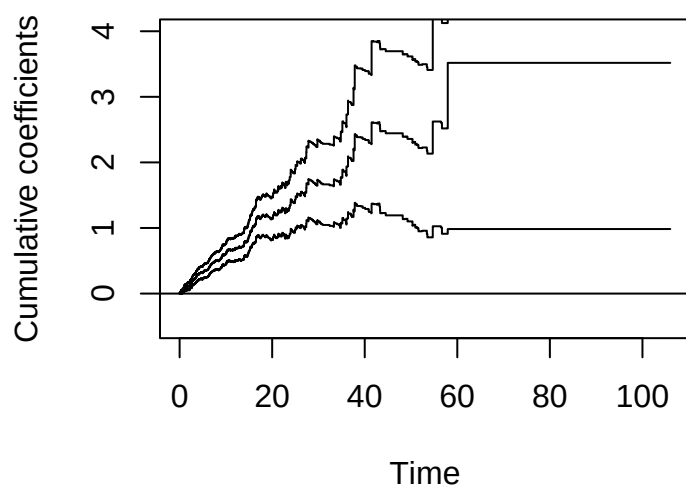
- Intercept: $p < 0.001 \rightarrow$ Baseline hazard clearly varies over time
- X_1 : $p < 0.001 \rightarrow$ Strong evidence that effect of X_1 changes over time
- X_2 : $p = 0.002 \rightarrow$ Strong evidence that effect of X_2 changes over time

- X_3 : $p=0.8 \rightarrow$ No strong evidence against constant effect
- X_4 : $p=0.081 \rightarrow$ No evidence of time variation

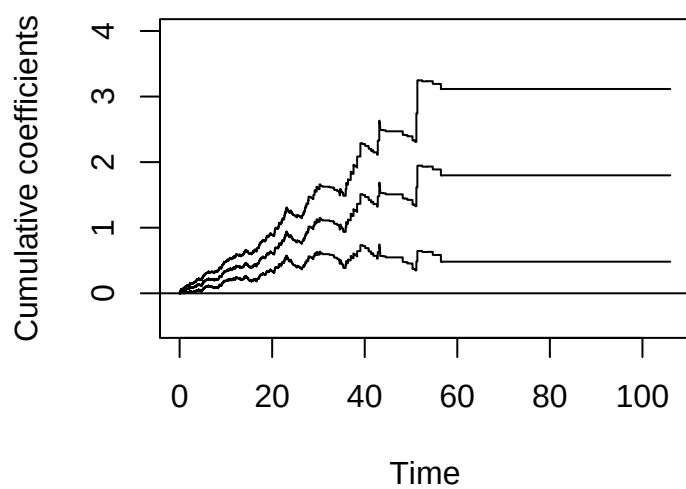
```
# Plot the non-parametric cumulative regression coefficients with 95% CI  
plot(aalen_fit, ylim = c(-0.5,4))
```



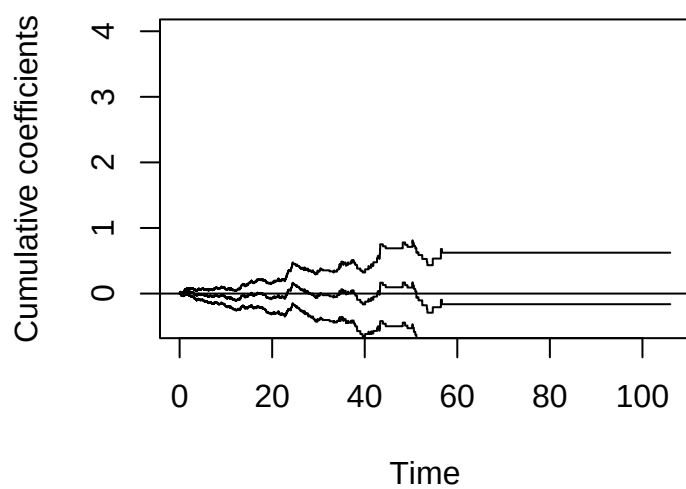
X1



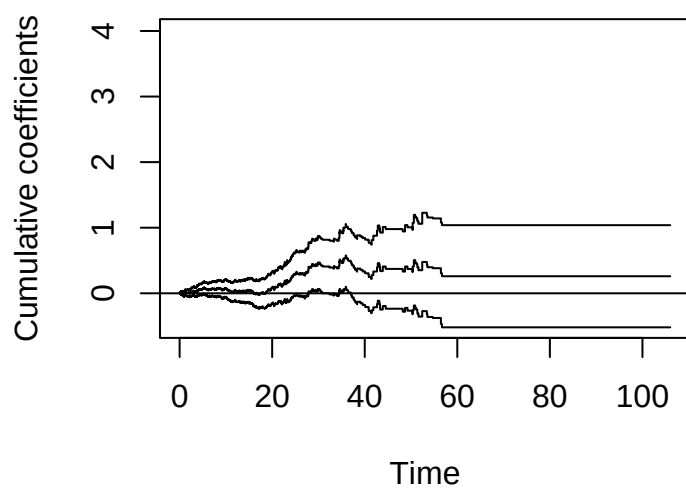
X2



X3



X4



We simulated $R = 1000$ datasets.

```

test_aalen <- replicate(R, {
  # Generate data
  data <- simulate_aalen_data(n, prev, sigma, lambda0, beta, c_max)
  # Estimate prevalences
  prev <- colMeans(data %>% dplyr::select(starts_with("X")))%>%
    stats::setNames(paste0("prev_", names(.)))
  # Censoring rate
  cens_rate <- stats::setNames(mean(1 - data$event), "cens_rate")
  # Fit Aalen model
  fit_obj <- timereg::aalen(Surv(time, event)~X1+X2+X3+X4, data)
  # Tests
  pval_supremum <- fit_obj[["pval.testBeq0"]]
  names(pval_supremum) <- paste0("pval_supremum_", names(pval_supremum))
  pval_KS <- fit_obj[["pval.testBeqC"]]
  names(pval_KS) <- paste0("pval_KS_", names(pval_KS))
  pval_CvM <- fit_obj[["pval.testBeqC.is"]]
  names(pval_CvM) <- paste0("pval_CvM_", names(pval_CvM))

  c(prev, cens_rate, pval_supremum, pval_KS, pval_CvM)
}, simplify = F)%>%
bind_rows

data.frame(colMeans(test_aalen))%>%
  tibble::rownames_to_column()%>%
  kable(col.names = c("Name", "Value"))

```

Name	Value
prev_X1	0.400025
prev_X2	0.399531
prev_X3	0.400150
prev_X4	0.400033
cens_rate	0.049713
pval_supremum_(Intercept)	0.000000
pval_supremum_X1	0.000000
pval_supremum_X2	0.030840
pval_supremum_X3	0.399380
pval_supremum_X4	0.487546
pval_KS_(Intercept)	0.000643
pval_KS_X1	0.002041
pval_KS_X2	0.060960
pval_KS_X3	0.354411

Name	Value
pval_KS_X4	0.419479
pval_CvM_(Intercept)	0.000932
pval_CvM_X1	0.000929
pval_CvM_X2	0.050253
pval_CvM_X3	0.378978
pval_CvM_X4	0.450623

```
test_aalen %>%
  dplyr::select(starts_with("pval"))%>%
  dplyr::mutate(across(everything(), ~ifelse(.x<0.05, 1, 0)))%>%
  dplyr::summarise_all(mean)%>%
  tidyr::pivot_longer(everything())%>%
  kable(col.names = c("p-value", "Rejection probability"))
```

p-value	Rejection probability
pval_supremum_(Intercept)	1.000
pval_supremum_X1	1.000
pval_supremum_X2	0.837
pval_supremum_X3	0.091
pval_supremum_X4	0.057
pval_KS_(Intercept)	0.999
pval_KS_X1	0.991
pval_KS_X2	0.726
pval_KS_X3	0.157
pval_KS_X4	0.090
pval_CvM_(Intercept)	0.997
pval_CvM_X1	0.995
pval_CvM_X2	0.752
pval_CvM_X3	0.128
pval_CvM_X4	0.082

3.2 Cox-Aalen model

The covariates are generated *independently* as follows:

$$X = (X_1, X_2, X_3, X_4)$$

where

- $X_1 \sim \mathcal{B}(0.4)$
- $X_2 \sim \mathcal{B}(0.4)$
- $X_3 \sim \mathcal{B}(0.4)$
- $X_4 \sim \mathcal{N}(0, 1)$

To generate mixed additive and multiplicative effects, event times were simulated under a Cox-Aalen model with constant baseline hazard: $\lambda_0(t) = 0.05$. Additive effects were assigned to X_1 and X_2 with coefficients $\beta_1 = 0.06$ and $\beta_3 = 0.005$, respectively. Multiplicative (proportional hazards) effects were assigned to X_3 and X_4 with coefficients $\beta_2 = 0.5$ and $\beta_4 = 0.2$, corresponding to hazard ratios of approximately 1.65 and 1.22.

```
n <- 1000
prev_add <- c(0.4, 0.4)
sigma_add <- diag(x = 1, nrow = 2, ncol = 2)
beta_add <- c(0.06, 0.005)
prev_mult <- 0.4
sigma_mult_bin <- diag(x = 1, nrow = 1, ncol = 1)
mean_mult <- 0
sigma_mult_cont <- diag(x = 1, nrow = 1, ncol = 1)
beta_mult <- c(0.5, 0.2)
lambda0 <- 0.05
target_cens <- 0.05
c_max <- calibrate_c_max_cox_aalen(lambda0          = lambda0,
                                   beta_add          = beta_add,
                                   beta_mult         = beta_mult,
                                   prev_add          = prev_add,
                                   sigma_add         = sigma_add,
                                   prev_mult         = prev_mult,
                                   mean_mult         = mean_mult,
                                   sigma_mult_bin     = sigma_mult_bin,
                                   sigma_mult_cont   = sigma_mult_cont,
                                   target_cens       = target_cens,
                                   seed              = 2026)

c_max
```

```
[1] 261.5929
```

```
data_cox_aalen <- simulate_cox_aalen_data(
  n, lambda0, beta_add, beta_mult,
  prev_add, sigma_add, prev_mult, mean_mult,
  sigma_mult_bin, sigma_mult_cont, c_max
)
head(data_cox_aalen)
```

	time	event	X1	X2	X3	X4
1	6.617109	1	1	1	0	-0.54856677
2	6.221355	1	0	1	0	-0.68120297
3	2.742523	1	1	1	1	0.33735955
4	3.837105	1	0	0	1	-0.29210344
5	3.260748	1	1	1	0	0.43125567
6	23.846538	1	0	0	0	-0.05632983

```
# Check prevalences
colMeans(data_cox_aalen %>% dplyr::select(X1:X3))
```

```
      X1      X2      X3
0.400 0.399 0.383
```

```
# Check mean and sd
round(mean(data_cox_aalen$X4), 3)
```

```
[1] 0
```

```
sd(data_cox_aalen$X4)
```

```
[1] 1
```

```
# Check censoring rate
mean(1 - data_cox_aalen$event)
```

```
[1] 0.043
```

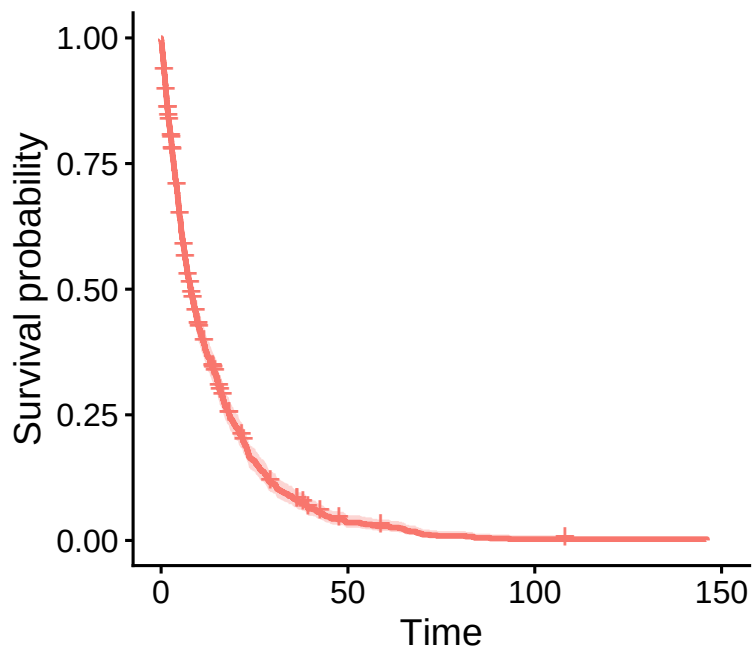
Kaplan-Meier

```
km_cox_aalen_fit <- survfit(Surv(time, event)~1, data_cox_aalen)
km_cox_aalen_fit
```

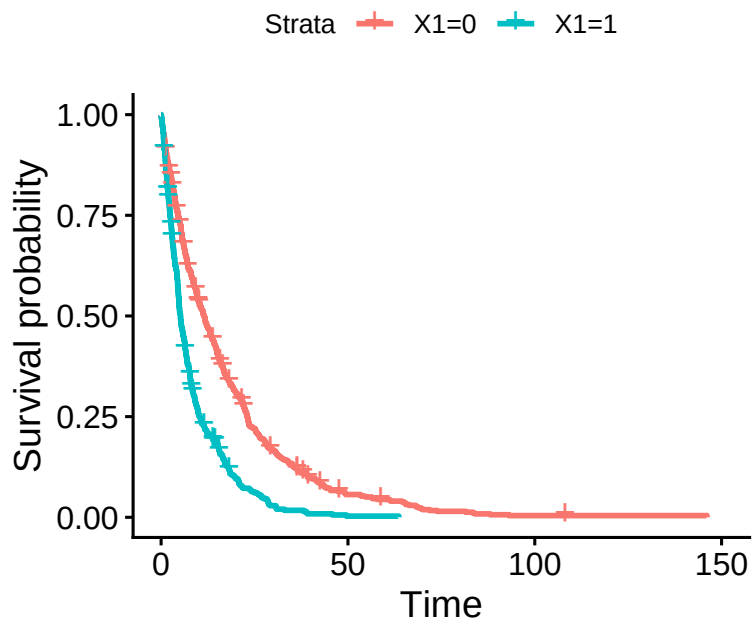
```
Call: survfit(formula = Surv(time, event) ~ 1, data = data_cox_aalen)
```

```
      n events median 0.95LCL 0.95UCL
[1,] 1000    957   7.98    7.05    8.81
```

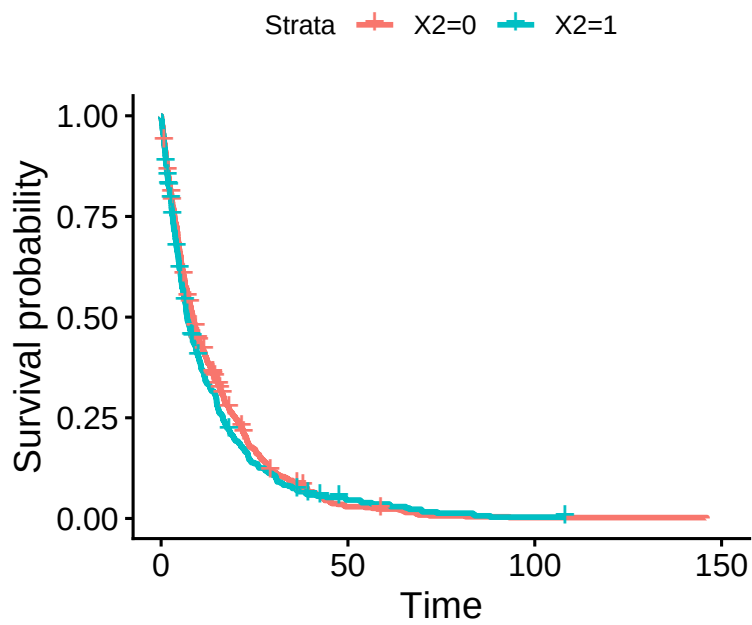
```
ggsurvplot(km_cox_aalen_fit, data = data_cox_aalen, legend = "none")
```



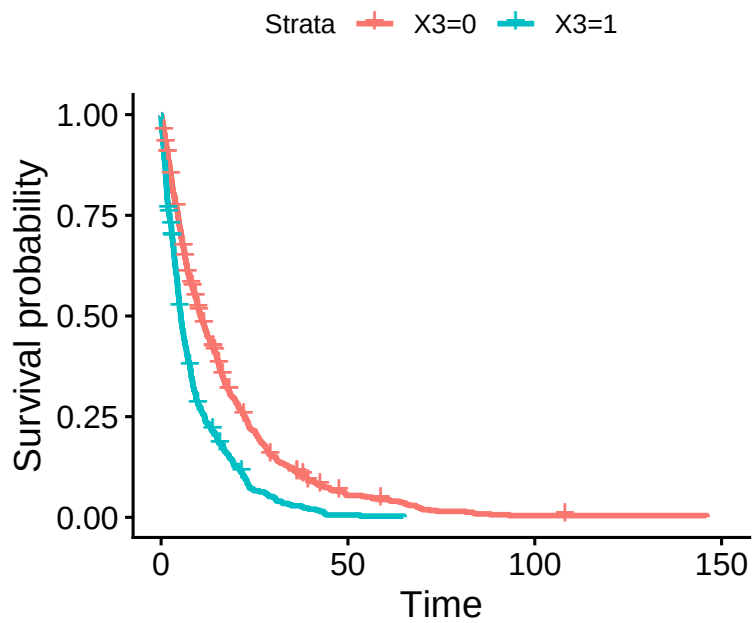
```
ggsurvplot(survfit(Surv(time, event)~X1, data_cox_aalen))
```



```
ggsurvplot(survfit(Surv(time, event)~X2, data_cox_aalen))
```



```
ggsurvplot(survfit(Surv(time, event)~X3, data_cox_aalen))
```



Cox-Aalen model

```
# Fit Cox-Aalen model
cox_aalen_fit <- timereg::cox.aalen(Surv(time, event)~X1+X2+prop(X3)+prop(X4),
                                   data = data_cox_aalen, n.sim = 1000, basesim = 1)
summary(cox_aalen_fit)
```

Cox-Aalen Model

Test for Aalen terms

Test for nonparametric terms

Test for non-significant effects

	Supremum-test of significance	p-value	H ₀ : B(t)=0
(Intercept)	13.50	0.000	
X1	8.82	0.000	
X2	2.85	0.096	

Test for time invariant effects

	Kolmogorov-Smirnov test	p-value	H ₀ : constant effect
(Intercept)	1.600	0.000	
X1	2.040	0.000	
X2	0.351	0.297	

Proportional Cox terms :

	Coef.	SE	Robust SE	D2log(L) ⁻¹	z	P-val	lower	2.5%
prop(X3)	0.673	0.0691	0.0672	0.0692	10.00	0.000000000000	0.538	
prop(X4)	0.197	0.0332	0.0320	0.0331	6.17	0.000000000701	0.132	

upper97.5%

prop(X3) 0.808

prop(X4) 0.262

Test of Proportionality

	sup	hat U(t)	p-value	H ₀
prop(X3)	12.8	0.334		
prop(X4)	24.2	0.413		

Test for non-significant effects

- Intercept: $p < 0.001 \rightarrow$ Baseline hazard is clearly non zero
- X_1 : $p < 0.001 \rightarrow$ Strong evidence that X_1 affects the hazard
- X_2 : $p = 0.10 \rightarrow$ No evidence that X_2 affects the hazard

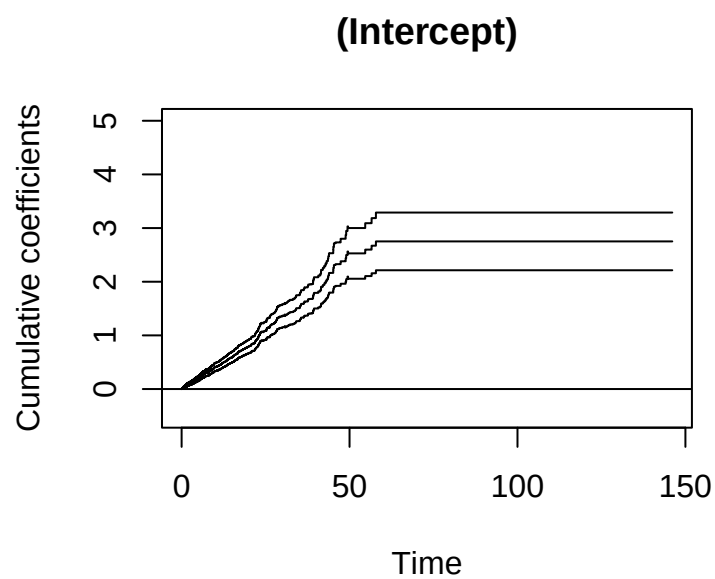
Test for time invariant effects

- Intercept: $p < 0.001 \rightarrow$ Baseline hazard clearly varies over time
- X_1 : $p < 0.001 \rightarrow$ Strong evidence that X_1 affects the hazard
- X_2 : $p = 0.3 \rightarrow$ No evidence of time variation

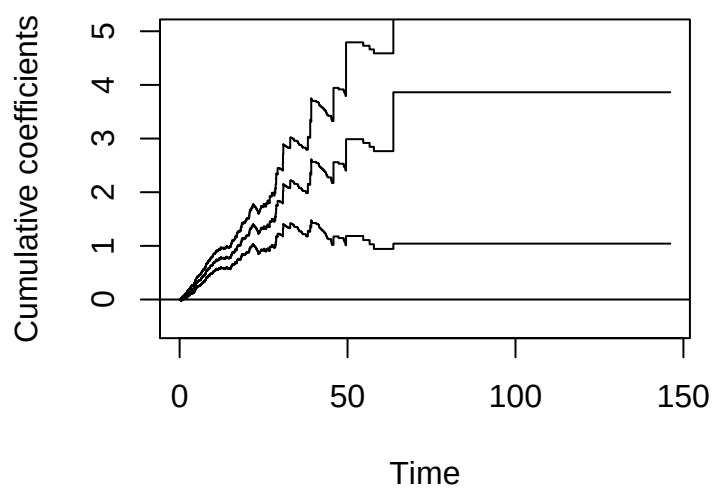
Test of proportionality

- X_3 : $p = 0.3 \rightarrow$ Compatible with a proportional effect
- X_4 : $p = 0.4 \rightarrow$ Compatible with a proportional effect

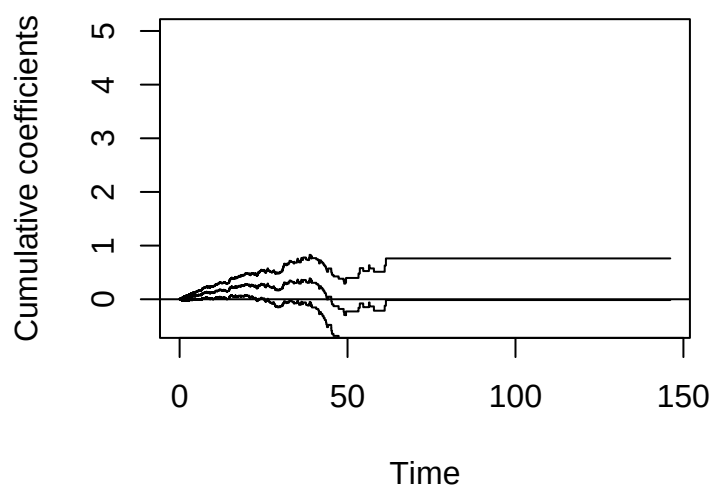
```
# Plot additive Aalen terms
plot(cox_aalen_fit, ylim = c(-0.5,5))
```



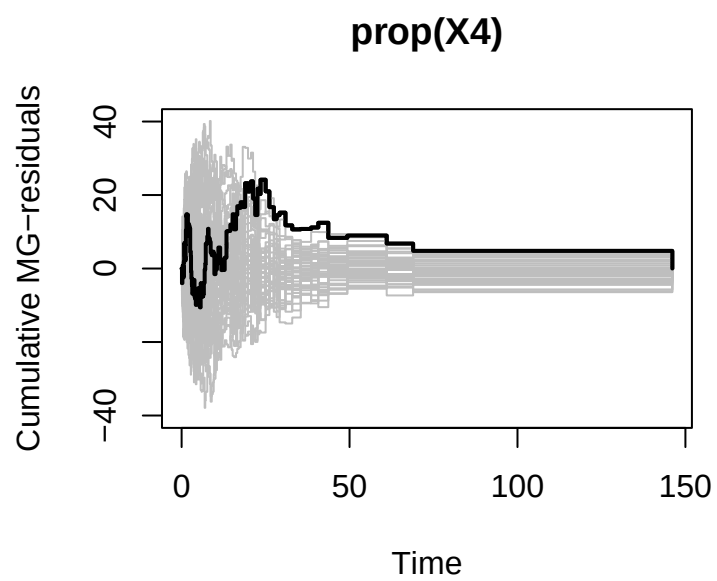
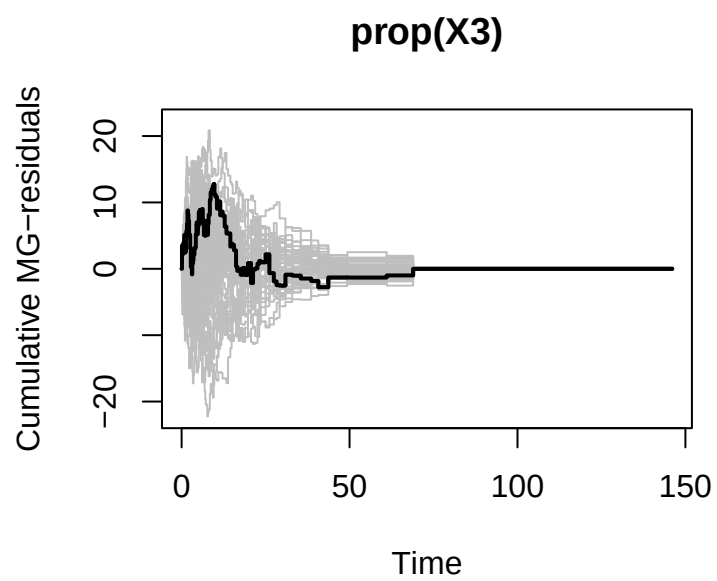
X1



X2



```
# Plot multiplicative terms  
plot(cox_aalen_fit, score= TRUE)
```



We simulated $R = 1000$ datasets.

```

test_cox_aalen <- replicate(R, {
  # Generate data
  data <- simulate_cox_aalen_data(n, lambda0, beta_add, beta_mult,
                                prev_add, sigma_add,
                                prev_mult, mean_mult,
                                sigma_mult_bin, sigma_mult_cont,
                                c_max)

  # Estimate prevalences
  prev <- colMeans(data %>% dplyr::select(X1:X3))%>%
    stats::setNames(paste0("prev_", names(.)))
  # Mean and sd X4
  mean_sd_X4 <- stats::setNames(c(mean(data$X4), sd(data$X4)),
                                c("mean_X4", "sd_X4"))

  # Censoring rate
  cens_rate <- stats::setNames(mean(1 - data$event), "cens_rate")
  # Fit Cox-Aalen model
  fit_obj <- timereg::cox.aalen(Surv(time, event)~X1+X2+prop(X3)+prop(X4),
                                data = data, n.sim = 1000, basesim = 1)

  # Additive terms
  pval_supremum <- fit_obj[["pval.testBeq0"]]
  names(pval_supremum) <- paste0("pval_supremum_", names(pval_supremum))
  pval_KS <- fit_obj[["pval.testBeqC"]]
  names(pval_KS) <- paste0("pval_KS_", names(pval_KS))
  # pval_CvM <- fit_obj[["pval.testBeqC.is"]]
  # names(pval_CvM) <- paste0("pval_CvM", names(pval_CvM))
  # Multiplicative terms
  coef <- coef(fit_obj)[,"Coef."]
  # Test for proportional
  pval_prop <- stats::setNames(
    fit_obj$pval.Prop,
    gsub("prop\\(|\\|\\|)", "", paste0("pval_prop_", names(fit_obj$pval.Prop)))
  )

  c(prev, mean_sd_X4, cens_rate,
    pval_supremum, pval_KS, #pval_CvM,
    coef, pval_prop)

}, simplify = F)%>%
bind_rows()

data.frame(round(colMeans(test_cox_aalen), 3))%>%
tibble::rownames_to_column()%>%

```

```
kable(col.names = c("Name", "Value"))
```

Name	Value
prev_X1	0.400
prev_X2	0.400
prev_X3	0.400
mean_X4	0.000
sd_X4	1.000
cens_rate	0.050
pval_supremum_(Intercept)	0.000
pval_supremum_X1	0.000
pval_supremum_X2	0.407
pval_KS_(Intercept)	0.007
pval_KS_X1	0.009
pval_KS_X2	0.387
prop(X3)	0.513
prop(X4)	0.189
pval_prop_X3	0.473
pval_prop_X4	0.413

```
test_cox_aalen %>%
  dplyr::select(starts_with("pval"))%>%
  dplyr::mutate(across(everything(), ~ifelse(.x<0.05, 1, 0)))%>%
  dplyr::summarise_all(mean)%>%
  tidyr::pivot_longer(everything())%>%
  mutate(value = round(value, 3))%>%
  kable(col.names = c("p-value", "Rejection probability"))
```

p-value	Rejection probability
pval_supremum_(Intercept)	1.000
pval_supremum_X1	1.000
pval_supremum_X2	0.099
pval_KS_(Intercept)	0.983
pval_KS_X1	0.964
pval_KS_X2	0.125
pval_prop_X3	0.052
pval_prop_X4	0.085